Flood Workbook Column Dictionary

This document explains the output columns used by the current flood-screening workbooks created by `src/check_points_against_jrc_floods.py`.

Current output files:

- `data/processed/T20_Anonymised_jrc_flood_check.xlsx`
- `data/processed/T20_Anonymised_gaspar_check.xlsx`

The goal is to keep the `event_hits` sheets short and readable, while still documenting every kept variable.

For the exact TRI and Copernicus riparian shapefiles behind the Gaspar spatial flags, see:

- `docs/tri_2020_sig_di_reference.md`

1. Workbook Structure

JRC workbook

Sheets:

- `point_flags`
- `Detailed`
- `candidate_events`
- `event_hits`

Meaning:

- `point_flags` gives one row per point and a simple `0/1` flood flag.
- `Detailed` keeps one row per original T20 input row, with all original workbook columns plus a leading binary `touched` flag.
- `candidate_events` keeps the full point x JRC-event diagnostic detail.
- `event_hits` keeps only the positive JRC matches and only the core columns.

Large-sheet note:

- if a workbook sheet exceeds the Excel row limit, the code automatically splits it into numbered tabs such as:
    - `candidate_events`
    - `candidate_events_2`
    - `candidate_events_3`

Gaspar workbook

Sheets:

- `point_flags`
- `Detailed`
- `candidate_events`

- `event_hits`

Meaning:

- `point_flags` gives one row per point and a simple `0/1` Gaspar-derived flood flag.
- `Detailed` keeps one row per original T20 input row, with all original workbook columns plus a leading binary `touched` flag.
- `candidate_events` keeps the point x Gaspar-event matches that survive the row-level date filter.
- `event_hits` keeps only the final spatial positives:
  - `TRI For`
  - or `outside n_tri` and `inside riparian`

Large-sheet note:

- if a workbook sheet exceeds the Excel row limit, the code automatically splits it into numbered tabs such as:
  - `candidate_events`
  - `candidate_events_2`
  - `candidate_events_3`

2. `point_flags` Columns

These columns are the same in both workbooks.

Column	Meaning
`point_id`	Point identifier from the input workbook.
`flag_flood`	Final point-level flag. `1` means at least one positive event hit was found for that point. `0` means none was found.

3. `Detailed` Sheet

These columns are written in both workbooks.

- `point_id` is moved to the front.
- `touched` is inserted right after it.
- all other original T20 columns are kept after those two leading fields.

Meaning of `touched`:

- JRC workbook: `1` means the point has at least one positive JRC event hit, where either the local `40 m` point buffer or the `1 km` surrounding buffer was flooded.
- Gaspar workbook: `1` means the point has at least one positive Gaspar hit after the `TRI For / outside n_tri + riparian` filter.

4. JRC `event_hits` Columns

The JRC `event_hits` sheet keeps only the essential point, event, date, and buffer-depth fields.

Column	Meaning
`point_id`	Point identifier from the input workbook.
`excel_row_number`	Original Excel row number of the point after header detection.
`point_latitude`	Point latitude used for the spatial check.
`point_longitude`	Point longitude used for the spatial check.
`lau_code`	Eurostat LAU code matched to the point.
`lau_name`	LAU name matched to the point.
`insee_com`	Current French commune INSEE code linked to the point.
`Reference_Date`	Raw point-level anchor date from the source workbook when present.
`Closed_Default_Date`	Preferred row-level end date from the source workbook when present.
`Cut_off_Date`	Fallback row-level end date from the source workbook when present.
`study_period_end`	Effective row-level end date actually used to filter JRC events.
`study_period_end_source`	Which source column supplied `study_period_end`.
`event_id`	JRC flood event identifier.
`raster_file`	TIFF filename used for the JRC event check.
`start_date`	JRC event start date.
`end_date`	JRC event end date.
`duration_days`	Duration of the JRC event in days.
`max_depth_cm`	Maximum flood depth reported by the processed JRC event table for the event.
`flooded_pixels`	Total flooded pixels reported by the processed JRC event table for the event.
`flooded_area_m2`	Total flooded area reported by the processed JRC event table for the event.
`hit_at_point`	`True` when the local `40 m` point buffer contains flooded pixels above threshold.
`exact_point_depth_cm`	Current local point-depth field written by the script for the `40 m` branch.
`point_buffer_radius_m`	Radius of the local buffer used around the point, in meters.
`point_buffer_flood_hit`	`True` when the `40 m` buffer contains flooded pixels above threshold.

Column	Meaning
`point_buffer_flooded_pixels`	Number of flooded pixels inside the `40 m` buffer.
`point_buffer_flooded_pixel_pct`	Share of pixels flooded inside the `40 m` buffer, in percent.
`point_buffer_flooded_area_m2`	Flooded area inside the `40 m` buffer, in square meters.
`point_buffer_min_depth_cm`	Minimum flooded-pixel depth inside the `40 m` buffer.
`point_buffer_max_depth_cm`	Maximum flooded-pixel depth inside the `40 m` buffer.
`point_buffer_median_depth_cm`	Median flooded-pixel depth inside the `40 m` buffer.
`point_buffer_mean_depth_cm`	Mean flooded-pixel depth inside the `40 m` buffer.
`buffer_radius_km`	Radius of the surrounding buffer used around the point, in kilometers.
`buffer_flood_hit`	`True` when the `1 km` surrounding buffer contains flooded pixels above threshold.
`buffer_flooded_pixels`	Number of flooded pixels inside the `1 km` surrounding buffer.
`buffer_flooded_pixel_pct`	Share of pixels flooded inside the `1 km` surrounding buffer, in percent.
`buffer_flooded_area_m2`	Flooded area inside the `1 km` surrounding buffer, in square meters.
`buffer_min_depth_cm`	Minimum flooded-pixel depth inside the `1 km` surrounding buffer.
`buffer_max_depth_cm`	Maximum flooded-pixel depth inside the `1 km` surrounding buffer.
`buffer_median_depth_cm`	Median flooded-pixel depth inside the `1 km` surrounding buffer.
`buffer_mean_depth_cm`	Mean flooded-pixel depth inside the `1 km` surrounding buffer.

5. Gaspar `candidate_events` and `event_hits` Columns

The Gaspar `event_hits` sheet is a filtered subset of the Gaspar `candidate_events` sheet.

`candidate_events` keeps all Gaspar point-event rows that survive the row-level date filter.

`event_hits` keeps only rows where:

- the point is in `TRI For`
- or the point is outside `n_tri` and inside a riparian polygon

The kept columns are:

Column	Meaning
`point_id`	Point identifier from the input workbook.
`excel_row_number`	Original Excel row number of the point after header detection.

Column	Meaning
`point_latitude`	Point latitude used for the spatial check.
`point_longitude`	Point longitude used for the spatial check.
`lau_code`	Eurostat LAU code matched to the point.
`lau_name`	LAU name matched to the point.
`insee_com`	Current French commune INSEE code linked to the point.
`Reference_Date`	Raw point-level anchor date from the source workbook when present.
`Closed_Default_Date`	Preferred row-level end date from the source workbook when present.
`Cut_off_Date`	Fallback row-level end date from the source workbook when present.
`study_period_end`	Effective row-level end date actually used to filter Gaspar events.
`study_period_end_source`	Which source column supplied `study_period_end`.
`gaspar_event_uid`	Unique Gaspar event-row identifier built in the processed Gaspar table.
`cod_nat_catnat`	CatNat decree identifier used in the Gaspar source.
`gaspar_start_date`	Gaspar event start date.
`gaspar_end_date`	Gaspar event end date.
`gaspar_commune_name`	Commune name attached to the Gaspar event row.
`gaspar_commune_match_method`	How the Gaspar commune was matched to the current point commune.
`tri_for_hit`	`True` when the point falls inside a plain TRI `For` polygon.
`tri_boundary_hit`	`True` when the point falls inside an `n_tri` territory-boundary polygon.
`tri_zone_status`	Simplified TRI status used by the Gaspar logic: `for`, `inside_n_tri_not_for`, or `outside_n_tri`.
`riparian_hit`	`True` when the point falls inside a riparian polygon.
`gaspar_hit_reason`	Final spatial reason kept by the workflow: `tri_for`, `riparian_outside_n_tri`, or `not_selected`.

6. Reading the Two Workbooks Together

The simplest interpretation is:

- JRC workbook:
  - `point_flags` answers: did JRC produce one or more positive flood hits for this point?

    - `event_hits` answers: which JRC events produced those hits, and what were the `40 m` and `1 km` depth metrics?
- Gaspar workbook:
    - `point_flags` answers: did Gaspar produce one or more final hits after the `TRI For / outside n_tri + riparian` rule?
    - `event_hits` answers: which Gaspar events survived both the row-level date filter and the final spatial rule?

7. Q&A

About `point_flags`

Question:

- does one row in `point_flags` mean one address, that is one latitude / longitude pair?

Answer:

- technically, one row in `point_flags` means one unique `point_id`
- in practice, if the source workbook has one address per row, that usually means one address = one point
- but the code does not force latitude / longitude uniqueness
- so two different rows can have the same latitude / longitude and still be treated as two distinct points if they have two different `point_id` values

About `Detailed`

Question:

- does `Detailed` also contain construction fields such as `study_period_anchor_date`?

Answer:

- yes
- `Detailed` keeps the original T20 row plus the extra study-period columns created by the script before export
- typical examples include:
    - `study_period_anchor_date`
    - `study_period_primary_end_date`
    - `study_period_fallback_end_date`
    - `study_period_start`
    - `study_period_end`
    - `study_period_end_source`

About JRC `event_hits`

Question:

- does `event_hits` contain one row per flood event before `study_period_end`?
- and for JRC, is a hit either `hit_at_point=True` or `buffer_flood_hit=True`?

Answer:

- yes, with one nuance
- in the current T20 setup, because the study logic keeps the full history up to `study_period_end`, the practical rule is:
  - `event_start_date <= study_period_end`
- more generally, the code uses interval-overlap logic
- and yes, a JRC event is kept in `event_hits` when it is positive locally, meaning:
  - `point_buffer_flood_hit = True`
  - or `surrounding_buffer_flood_hit = True`
- `buffer_flood_hit` is the simplified-sheet alias for the `1 km` surrounding-buffer logic

About `lau_code`

Question:

- what LAU level is `lau_code`?

Answer:

- it is the Eurostat LAU level at municipality / commune scale
- for France, it can be read as a commune-level LAU code such as `FR_25011`

About `flooded_pixels`

Question:

- is `flooded_pixels` the total number of flooded pixels in the whole JRC TIFF?

Answer:

- no
- it is not the total for the full TIFF
- `flooded_pixels` is the number of flooded pixels for that event inside the LAU attached to the row
- so it is:
  - not limited to the `40 m` buffer
  - not limited to the `1 km` buffer
  - not the whole TIFF either
- it is an `event x LAU` metric precomputed in the processed JRC event table
- for local point-neighborhood metrics, use:
  - `point_buffer_flooded_pixels`
  - `buffer_flooded_pixels`

About `point_buffer_radius_m` and `buffer_radius_km`

Question:

- are they always equal to `40 m` and `1 km` in the current methodology?

Answer:

- yes, with the current run settings
- more precisely:
  - `point_buffer_radius_m = 40`
  - `buffer_radius_km = 1`
- these are script parameters, so they change only if the CLI values are changed at runtime

About `hit_at_point` and `point_buffer_flood_hit`

Question:

- what is the difference between `hit_at_point` and `point_buffer_flood_hit`?

Answer:

- in the current version, there is no functional difference
- `hit_at_point` is a historical alias
- it now reflects the same logic as `point_buffer_flood_hit`
- in practice:
  - `hit_at_point == point_buffer_flood_hit`
- the old name may sound like an exact pixel under the coordinate, but the real current logic is the local `40 m` buffer

About `gaspar_commune_match_method`

Question:

- what are the possible values of `gaspar_commune_match_method`?

Answer:

- the current code can produce:
  - `current_code_exact`
  - `historical_code_update_ready`
  - `current_name_unique_adminexpress`
  - `current_name_unique_lau`
- and in internal diagnostics it may also show:
  - `unresolved`
- but the final point-matching workflow normally keeps resolved rows

About `tri_zone_status`

Question:

- is the following rule correct?
  - if `tri_for_hit=True` then `tri_zone_status="for"`
  - if `tri_for_hit=False` and `tri_boundary_hit=True` then `tri_zone_status="inside_n_tri_not_for"`
  - if `tri_for_hit=False` and `tri_boundary_hit=False` then `tri_zone_status="outside_n_tri"`

Answer:

- yes, that is correct
- one nuance:
  - if `tri_for_hit=True`, that status takes priority even if the point is also inside the broader `n_tri` boundary

About `gaspar_hit_reason`

Question:

- is the following rule correct?
  - if `tri_for_hit=True` then `gaspar_hit_reason="tri_for"`
  - if `tri_for_hit=False` and `tri_boundary_hit=False` and `riparian_hit=True` then `gaspar_hit_reason="riparian_outside_n_tri"`
  - otherwise `gaspar_hit_reason="not_selected"`

Answer:

- yes, that is correct
- so the following cases correctly lead to `not_selected`:
  - `tri_for_hit=False` and `tri_boundary_hit=True`
  - `tri_for_hit=False` and `tri_boundary_hit=False` and `riparian_hit=False`